

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1104

COURSE OUTCOME COVERED

CO1: *Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)*

Assessment Tool Weightage: 30%

Dr A Gayathri

QUESTIONS: 6

Total Marks:30

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I, Judy Allen J (192520055) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Judy Allen J

1. Suppose a software system is developed for a Smart Banking System where customers can open accounts, transfer funds, and pay utility bills.

(a) Explain the following object-oriented concepts:

- Class
- Object
- Message Passing
- Encapsulation
- Polymorphism

(b) Describe the following object relationships:

- Association
- Aggregation
- Composition

(c) Explain how these concepts improve software reusability and maintainability in a banking system.

Answer:

1(a)

Class

A class is a blueprint or template used to create objects. It defines the attributes (data) and methods (functions) that the objects will possess. A class does not occupy memory until an object is created.

Example:

In a Smart Banking System, Customer is a class with attributes such as customer ID, name, account number, and balance, and methods such as deposit(), withdraw(), and transfer().

Object

An object is a real-world entity created from a class. It contains actual values for the attributes and can perform the operations defined by the class.

Example:

A customer named Rahul with account number 123456789 is an object of the Customer class. Rahul can deposit money, withdraw cash, and transfer funds.

Message Passing

Message passing is the process through which one object communicates with another by invoking its methods. It enables collaboration among objects while maintaining independence.

Example:

When Rahul transfers ₹5,000 to Priya, the Customer object sends a message to the Account object to deduct the amount and another message to Priya's Account object to credit the amount.

Encapsulation

Encapsulation is the process of wrapping data and methods together into a single unit while restricting direct access to the data. Access is provided through public methods.

Example:

The account balance is kept private and can only be accessed through methods like deposit(), withdraw(), and getBalance(), preventing unauthorized modification.

Polymorphism

Polymorphism allows a single method to perform different tasks depending on the object that invokes it. It improves flexibility and code reuse.

Example:

The method calculateInterest() may produce different results for Savings Account, Current Account, and Fixed Deposit Account because each account type follows different interest rules.

1(b)**Association**

Association is a relationship where two independent objects communicate with each other while maintaining their own existence.

Example:

A Customer is associated with a Bank Account because the customer performs transactions using the account.

Aggregation

Aggregation is a "has-a" relationship where one object contains another object, but the contained object can exist independently.

Example:

A Bank contains many Customers. Even if one branch is closed, customer information can still exist in another branch.

Composition

Composition is a strong "part-of" relationship where the contained object cannot exist independently if the parent object is destroyed.

Example:

A Bank Account contains Transaction records. If the account is permanently deleted, all associated transaction records are also removed.

1(c)

Object-oriented concepts make the Smart Banking System more reusable and maintainable by organizing the software into independent classes and objects.

- Classes allow developers to reuse existing code for multiple customers and accounts.
- Encapsulation protects sensitive banking information such as account balances and passwords from unauthorized access.
- Polymorphism enables the same method to work for different account types without rewriting code.
- Association, Aggregation, and Composition clearly define relationships between banking entities, making the system easier to understand and modify.
- Message Passing enables secure communication between objects while keeping modules independent.

As a result, the banking system becomes more secure, scalable, easy to maintain, and easier to upgrade with new features such as loan management, online payments, and mobile banking.

2. Consider developing a Smart Healthcare Management System.

(a) Explain the following object-oriented design principles:

- Abstraction
- Encapsulation
- Modularity

(b) Describe the following SOLID principles:

- Liskov Substitution Principle (LSP)
- Interface Segregation Principle (ISP)

(c) Explain how these principles improve software quality and flexibility.

Answer:

2(a)

Abstraction

Abstraction is the process of showing only the essential features of an object while hiding unnecessary implementation details. It helps users focus on what an object does instead of how it works.

Example:

In a Smart Healthcare Management System, a doctor can view patient records and prescribe medicines without knowing how the database stores the information.

Encapsulation

Encapsulation is the process of combining data and methods into a single unit (class) and restricting direct access to the data. Private data can only be accessed through public methods.

Example:

The patient's medical history is kept private and can only be accessed through authorized methods such as `viewRecord()` or `updateRecord()`, ensuring data security.

Modularity

Modularity is the process of dividing a software system into smaller, independent modules, where each module performs a specific function. This makes the system easier to develop, test, maintain, and reuse.

Example:

A Smart Healthcare Management System can be divided into separate modules such as Patient Management, Doctor Management, Appointment Scheduling, Billing, Pharmacy, and Laboratory. Each module works independently but together forms the complete system.

2(b)

Liskov Substitution Principle (LSP)

The Liskov Substitution Principle states that a subclass should be able to replace its parent class without changing the correctness of the program.

Example:

If Doctor is the parent class and Cardiologist and Neurologist are subclasses, any module expecting a Doctor object should work correctly with either specialist without modifying the program.

Interface Segregation Principle (ISP)

The Interface Segregation Principle states that a class should not be forced to implement methods that it does not use. Instead of one large interface, multiple smaller and specific interfaces should be created.

Example:

In a healthcare system:

- A Doctor interface contains methods like `diagnosePatient()` and `prescribeMedicine()`.
- A Receptionist interface contains methods like `registerPatient()` and `scheduleAppointment()`.

This prevents unnecessary implementation of unrelated methods.

2(c)

The object-oriented design principles improve the quality and flexibility of a Smart Healthcare Management System in several ways:

- Abstraction simplifies the system by hiding internal implementation details, making it easier for users to interact with the software.
- Encapsulation protects sensitive patient information by restricting unauthorized access and improving data security.
- Modularity divides the application into independent modules, making development, testing, debugging, and maintenance easier.
- Liskov Substitution Principle (LSP) allows new doctor or staff types to be added without affecting existing system functionality, improving extensibility.
- Interface Segregation Principle (ISP) ensures that each class implements only the methods it requires, reducing complexity and improving maintainability.

Overall, these principles make the Smart Healthcare Management System secure, scalable, reusable, flexible, easy to maintain, and capable of accommodating future enhancements without major changes.

3. Suppose a software company is developing an Online Food Delivery System.

(a) Explain the phases of the Software Development Life Cycle (SDLC):

- Planning
- Requirement Analysis
- Design
- Development
- Testing
- Maintenance

(b) Describe the purpose of each SDLC phase.

(c) Explain how SDLC contributes to delivering reliable software.

Answer:

3(a)

1. Planning

Planning is the first phase of the SDLC. In this phase, the project goals, scope, budget, schedule, and required resources are identified. The feasibility of the project is also analyzed to determine whether it can be successfully completed.

Example:

For an Online Food Delivery System, the development team plans features such as user registration, restaurant listings, food ordering, online payment, and order tracking.

2. Requirement Analysis

In this phase, developers collect and analyze the functional and non-functional requirements from customers and stakeholders. These requirements are documented in the Software Requirement Specification (SRS).

Example:

The system should allow customers to browse restaurants, place food orders, make secure online payments, track deliveries, and provide ratings and reviews.

3. Design

The design phase creates the blueprint of the software. It includes system architecture, database design, user interface design, and UML diagrams to define how the system will be developed.

Example:

The Online Food Delivery System is designed with modules such as Customer, Restaurant, Delivery Partner, Payment Gateway, and Order Management.

4. Development

In this phase, programmers write the source code according to the design specifications. Each module is developed separately and integrated to build the complete application.

Example:

Developers create features such as customer login, restaurant menu display, order placement, payment processing, and delivery tracking.

5. Testing

Testing verifies that the software works correctly and meets all specified requirements. Errors and bugs are identified and corrected before deployment.

Example:

The testing team checks whether orders are placed correctly, payments are processed securely, and delivery status is updated accurately.

6. Maintenance

Maintenance is the final phase of the SDLC. After deployment, developers fix bugs, improve performance, add new features, and update the software according to user feedback.

Example:

New features such as live order tracking, discount coupons, and multiple payment options are added after the application is launched.

3(b)

- Planning: Defines project objectives, scope, cost, timeline, and feasibility.
- Requirement Analysis: Collects and documents customer requirements to ensure the software meets user needs.
- Design: Creates the overall architecture and structure of the software before coding begins.
- Development: Converts the design into working software through programming.
- Testing: Detects and fixes defects to ensure the software functions correctly and reliably.
- Maintenance: Keeps the software updated by fixing issues, improving performance, and adding new features after deployment.

3(c)

The Software Development Life Cycle (SDLC) provides a structured and systematic approach to software development. Each phase has a specific objective, ensuring that the project progresses in an organized manner.

- Proper planning reduces project risks and resource wastage.
- Requirement analysis ensures that customer needs are accurately understood and documented.

- A well-prepared design provides a strong foundation for development.
- Development follows standardized coding practices, improving software quality.
- Thorough testing identifies and removes defects before deployment.
- Continuous maintenance keeps the software secure, efficient, and up to date.

For an Online Food Delivery System, following the SDLC ensures that the application is reliable, secure, user-friendly, scalable, easy to maintain, and capable of providing accurate food ordering, payment processing, and delivery services.

4. Consider a project for developing a Smart Agriculture Monitoring System, where customer requirements change regularly.

(a) Explain the following SDLC models:

- Prototype Model
- Spiral Model
- Agile Model
- DevOps Model

(b) Compare the Prototype Model and Agile Model.

(c) Recommend the most suitable SDLC model for this project with justification.

Answer:

4(a)

1. Prototype Model

The Prototype Model is an SDLC model in which a working prototype (sample version) of the software is developed before the final system. The prototype allows users to review the system, provide feedback, and suggest changes. Based on this feedback, the prototype is refined until the final software meets user requirements.

Example:

In a Smart Agriculture Monitoring System, an initial prototype may display soil moisture, temperature, and humidity data. Farmers can test it and request additional features such as weather forecasting or automatic irrigation before the final system is developed.

2. Spiral Model

The Spiral Model combines iterative development with risk analysis. The software is developed through multiple cycles (spirals), and each cycle includes planning, risk analysis, development, and evaluation. This model is suitable for large and complex projects with high risks.

Example:

A Smart Agriculture Monitoring System that integrates IoT sensors, drones, and AI-based crop analysis can use the Spiral Model to identify and reduce technical risks during each development cycle.

3. Agile Model

The Agile Model develops software in small iterations called sprints. Each sprint delivers a working version of the software, allowing continuous customer feedback and quick adaptation to changing requirements.

Example:

If farmers request additional features such as pest detection or fertilizer recommendations during development, these can be added in the next sprint without affecting the entire project.

4. DevOps Model

The DevOps Model integrates software development (Dev) and IT operations (Ops). It focuses on continuous integration, continuous testing, continuous deployment, and continuous monitoring to deliver software quickly and reliably.

Example:

Updates to the Smart Agriculture Monitoring System, such as improved irrigation algorithms or bug fixes, can be automatically tested and deployed with minimal downtime.

4(b)

Prototype Model	Agile Model
Develops a prototype before the final software.	Develops software in small working iterations called sprints.
Used when requirements are unclear or incomplete.	Used when requirements change frequently during development.
Customer feedback is mainly collected after reviewing the prototype.	Customer feedback is collected after every sprint.
Focuses on refining the prototype until requirements are finalized.	Focuses on continuous development and regular software delivery.
Suitable for understanding customer needs before development begins.	Suitable for projects requiring flexibility and rapid updates.

4(c)

The Agile Model is the most suitable SDLC model for developing a Smart Agriculture Monitoring System because customer requirements change regularly.

Justification:

- Farmers may request new features such as weather prediction, crop disease detection, or automated irrigation during development.
- Agile allows these changing requirements to be incorporated quickly through short development cycles (sprints).
- Continuous customer feedback ensures the software meets real user needs.
- Frequent testing in every sprint improves software quality and reduces defects.
- Working software is delivered regularly, enabling early use and continuous improvement.
- Agile supports flexibility, faster development, and easier maintenance compared to traditional SDLC models.

Therefore, the Agile Model provides a flexible, customer-oriented, and efficient approach for developing a Smart Agriculture Monitoring System where requirements evolve over time.

5. Suppose a software analyst is designing a Smart Transport Management System.

(a) Explain the importance of UML in object-oriented software development.

(b) Describe the following UML diagrams:

- State Diagram
- Component Diagram
- Deployment Diagram
- Package Diagram

(c) Explain how UML models improve communication among stakeholders.

Answer:

5(a)

Unified Modeling Language (UML) is a standardized graphical language used to visualize, design, document, and communicate the structure and behavior of an object-oriented software system before coding begins.

Importance of UML:

- UML provides a clear visual representation of the software system.
- It helps developers understand the system requirements and overall design.
- It improves communication among developers, designers, customers, and stakeholders.
- UML identifies design errors and inconsistencies before implementation, reducing development costs.
- It supports object-oriented concepts such as classes, objects, inheritance, association, and composition.
- It serves as proper documentation, making the software easier to maintain and upgrade.
- UML enables better planning, resulting in improved software quality and reduced development time.

Example:

In a Smart Transport Management System, UML diagrams illustrate the interactions between passengers, drivers, buses, payment systems, and administrators, helping developers understand the complete workflow before coding.

5(b)

1. State Diagram

A State Diagram represents the different states of an object during its life cycle and the transitions between those states based on events.

Example:

In a Smart Transport Management System, a bus ticket may move through the following states:

Booked → Confirmed → Used → Expired

The diagram helps developers understand how an object changes its state over time.

2. Component Diagram

A Component Diagram illustrates the organization of software components and their dependencies. It shows how different modules interact to form the complete system.

Example:

The Smart Transport Management System may contain the following components:

- Passenger Module
- Driver Module
- Vehicle Management Module
- Payment Module
- GPS Tracking Module
- Database Server

The diagram shows how these components communicate with each other.

3. Deployment Diagram

A Deployment Diagram shows the physical architecture of the system, including hardware devices, servers, databases, and network connections where the software is deployed.

Example:

In the Smart Transport Management System:

- User Mobile Application
- Web Server
- Application Server
- Database Server
- GPS Devices installed in buses

The deployment diagram illustrates how these hardware and software components are connected.

4. Package Diagram

A Package Diagram groups related classes into packages to organize the software into manageable modules. It shows dependencies between packages and improves software organization.

Example:

The Smart Transport Management System may have the following packages:

- User Management
- Booking Management
- Vehicle Management
- Payment Management
- Report Management
- Notification Management

This modular organization simplifies software development and maintenance.

5(c)

UML models improve communication by providing a common visual language that is easy for developers, analysts, designers, testers, managers, and customers to understand.

- UML diagrams clearly represent the structure and behavior of the system.
- Developers can understand the software architecture before coding begins.
- Customers can easily verify whether the proposed system satisfies their requirements.
- Testers can design effective test cases based on UML models.
- Project managers can monitor project progress and coordinate development activities.
- UML documentation simplifies future maintenance, debugging, and system enhancements.

For a Smart Transport Management System, UML ensures that all stakeholders have a clear understanding of system functionality, reducing misunderstandings and improving collaboration. This results in the development of reliable, scalable, maintainable, and high-quality software.

6. Consider developing a Smart Inventory Management System.

(a) Explain the concept of modular software design.

(b) Compare the procedural programming approach with the object-oriented programming approach.

(c) Explain how modularity improves software scalability, maintainability, and reusability

Answer:

6(a)

Modular software design is a software development approach in which a large application is divided into smaller, independent modules. Each module performs a specific function and can be developed, tested, maintained, and reused independently. The modules work together to form the complete software system.

Example:

In a Smart Inventory Management System, the software can be divided into the following modules:

- Product Management Module
- Supplier Management Module
- Inventory Tracking Module
- Order Management Module
- Billing Module
- Report Generation Module

Each module performs its own task while communicating with other modules whenever necessary.

6(b)

Procedural Programming	Object-Oriented Programming (OOP)
Focuses on functions and procedures.	Focuses on objects and classes.
Data and functions are separate.	Data and methods are combined into a single unit (class).
Uses a top-down approach.	Uses a bottom-up approach.
Provides less security because data is easily accessible.	Provides better security through encapsulation and data hiding.
Code reusability is limited.	Supports code reusability through inheritance and polymorphism.
Difficult to maintain large software systems.	Easier to maintain and extend large software systems.
Suitable for small applications.	Suitable for large and complex applications.

Example:

In a Smart Inventory Management System, OOP represents entities such as Product, Supplier, and Order as objects, making the software easier to organize and maintain. In procedural programming, all operations are written as separate functions, making large systems harder to manage.

6(c)**Modularity provides several advantages in software development:****Scalability**

Each module can be expanded independently without affecting the entire system.

Example:

A new Barcode Scanning Module or Warehouse Management Module can be added to the Smart Inventory Management System without modifying existing modules.

Maintainability

Since each module is independent, developers can easily identify and fix bugs or update features without disturbing other parts of the system.

Example:

If an error occurs in the Billing Module, only that module needs to be modified while the remaining modules continue to function normally.

Reusability

Modules developed for one project can be reused in other software applications, reducing development time and cost.

Example:

The Authentication Module used in the Smart Inventory Management System can also be reused in an E-Commerce System or Hospital Management System.

Overall Benefits

Modular software design improves software quality by making the system:

- Easy to develop and test.
- Easy to maintain and debug.
- Highly scalable for future enhancements.
- Reusable across different projects.
- More reliable and organized.
- Faster to update with new features.

Therefore, modular design helps build a Smart Inventory Management System that is efficient, flexible, maintainable, scalable, and reusable, resulting in better software quality and reduced development effort.